Lab 36 — Network IDS with Suricata

> **SY0-701 Objective 4.5** — Given a scenario, modify enterprise capabilities to enhance security
>
> **Domain 4** — Security Operations (28%) · **Time:** ~20 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Run Suricata as a network intrusion detection system, load signatures, write your own rules and read the alerts it raises in `eve.json`.

You'll build: A Suricata sensor watching the loopback interface, a custom rule file with three signatures (content match, web-attack pattern, scan threshold), and an alert report built with `jq`.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; tools installed in step 1. Lab 20 (firewall) helps but is not required.

Background

- An **IDS** inspects traffic and **alerts**; an **IPS** sits **inline** and can **drop** — the same Suricata rules can do either, depending on how the sensor is placed (tap/monitor vs inline).
- **Signature-based** detection matches known patterns (content, URI, user agent); **anomaly/behaviour-based** detection flags deviations such as a burst of SYN packets.
- A Suricata rule is `action protocol source -> destination (options; sid; rev;)`. The **sid** uniquely identifies the rule; local rules use sids above 1,000,000.
- Alerts are written as JSON lines to `/var/log/suricata/eve.json` — the feed a SIEM (Lab 33) ingests.
- Tuning matters: too many **false positives** cause alert fatigue; **false negatives** are missed attacks.

Steps

1. Install Suricata and helpers

Suricata is in the Ubuntu archive; `jq` reads the JSON alerts and `nmap` generates scan traffic against your own host.

```
apt-get update -qq && apt-get install -y suricata jq nmap curl python3
suricata --build-info | grep -E "^This is Suricata|AF_PACKET"
```

2. Download the open ruleset

`suricata-update` fetches the free Emerging Threats Open ruleset into `/var/lib/suricata/rules/suricata.rules`.

```
suricata-update -q || echo "Ruleset download failed — the custom rules in step 4 still work offline"
```

```
ls -lh /var/lib/suricata/rules/ 2>/dev/null
grep -c "^alert" /var/lib/suricata/rules/suricata.rules 2>/dev/null || true
```

3. Start a local web server to protect

All test traffic stays on this playground. The server listens on port 8080 on loopback.

```
mkdir -p /srv/lab36 && echo "Lab 36 web app" > /srv/lab36/index.html
cd /srv/lab36 && nohup python3 -m http.server 8080 --bind 127.0.0.1 >/tmp/web36.log 2>&1 &
sleep 1; curl -s http://127.0.0.1:8080/
```

4. Write custom signatures

Three local rules: a forbidden admin path, a SQL-injection pattern in the URI, and a threshold rule that fires when one source sends many SYN packets in a short window (a port scan).

```
mkdir -p /etc/suricata/rules
cat > /etc/suricata/rules/lab36.rules <<'EOF'
alert http any any -> any any (msg:"LAB36 access to restricted admin path";
flow:established,to_server; http.uri; content:"/secret-admin"; nocase; classtype:policy-
violation; sid:1000001; rev:1;)
alert http any any -> any any (msg:"LAB36 SQL injection pattern in URI";
flow:established,to_server; http.uri; content:"union select"; nocase; classtype:web-
application-attack; sid:1000002; rev:1;)
alert tcp any any -> any any (msg:"LAB36 possible TCP SYN port scan"; flags:S,12;
threshold:type both, track by_src, count 20, seconds 10; classtype:attempted-recon;
sid:1000003; rev:1;)
EOF
suricata -T -c /etc/suricata/suricata.yaml -s /etc/suricata/rules/lab36.rules -v 2>&1 |
tail -3
```

5. Run Suricata in IDS mode on loopback

`-i lo` sniffs the loopback interface, `-k none` disables checksum validation (loopback packets carry offloaded checksums), and `-s` adds the lab rules alongside the configured ruleset.

```
rm -f /var/log/suricata/eve.json
suricata -c /etc/suricata/suricata.yaml -i lo -k none -s /etc/suricata/rules/lab36.rules -
l /var/log/suricata -D
sleep 15
tail -2 /var/log/suricata/suricata.log
```

```
... All AFP capture threads are running.
```

6. Generate suspicious traffic against your own host

Request the restricted path, send a SQL-injection-style query string, then run a fast SYN scan of localhost.

```
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8080/secret-admin
curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1:8080/search?
q=1%20UNION%20SELECT%20password%20FROM%20users"
```

```
nmap -sS -p 1-200 -T4 127.0.0.1 >/dev/null
sleep 5
```

7. Read and summarise the alerts

Every alert is one JSON line. Filter on `event_type`, then count by signature — the same triage an analyst does in a SIEM.

```
jq -c 'select(.event_type=="alert") | {time:.timestamp, sid:.alert.signature_id,
sig:.alert.signature, src:.src_ip, dport:.dest_port}' /var/log/suricata/eve.json | tail -5
jq -r 'select(.event_type=="alert") | .alert.signature' /var/log/suricata/eve.json | sort
| uniq -c | sort -rn
```

8. Tune a noisy rule and compare IDS with IPS

Suppress an alert for a trusted source instead of deleting the rule, then note what an inline IPS deployment would change.

```
cat >> /etc/suricata/threshold.config <<'EOF'
# Lab 36 tuning: ignore scans from a trusted vulnerability scanner address
suppress gen_id 1, sig_id 1000003, track by_src, ip 10.0.0.50
EOF
grep -n "1000003" /etc/suricata/threshold.config
grep -nE "^\s*#?\s*(copy-mode|mode): " /etc/suricata/suricata.yaml | head -5
```

In IPS mode the sensor runs inline (AF_PACKET `copy-mode: ips` or NFQUEUE) and rules written with the `drop` action block the packet instead of only alerting.

Test it

```
jq -r 'select(.event_type=="alert") | .alert.signature_id' /var/log/suricata/eve.json |
sort -u | grep -E "^100000[123]$"
```

Expected: the three local sids `1000001`, `1000002` and `1000003` are listed, proving the content rule, the web-attack rule and the threshold (scan) rule all fired.

Troubleshooting

Symptom	Fix
No alerts at all, but `eve.json` has `flow` events	Checksum offload — make sure Suricata was started with `-k none`; restart with `pkill suricata` then re-run step 5.
`eve.json` does not exist	Suricata is still starting; wait for "All AFP capture threads are running" in `/var/log/suricata/suricata.log`, or check `-T` output in step 4 for a rule syntax error.
Scan rule (1000003) does not fire	Raw SYN scans need root (`whoami`); widen the scan to `-p 1-1000` so more than 20 SYNs arrive within 10 seconds.

Challenge

Add a rule that alerts on any HTTP request whose user agent contains `sqlmap`, prove it with `curl -A "sqlmap/1.7" http://127.0.0.1:8080/`, then rewrite it with the `drop` action and explain why it only blocks when Suricata runs inline.

Reflection

Your scan rule uses a threshold rather than a single packet: which is more likely to produce false positives in a busy network, and how would you tune it without creating false negatives?

Exam tips

- IDS = passive (tap/monitor, alerts only); IPS = inline (can drop). An inline device that fails can be configured **fail-open** (availability) or **fail-closed** (security).
- Signature-based detection misses zero-days; anomaly/heuristic detection catches new behaviour but raises more false positives.
- A **false positive** is an alert on benign activity; a **false negative** is an attack that raised no alert — the more dangerous of the two.

References

- Suricata documentation — https://docs.suricata.io
- Emerging Threats Open ruleset — https://rules.emergingthreats.net
- MITRE D3FEND network traffic analysis — https://d3fend.mitre.org

---